

# I3DP: Neuro-Symbolic Inductive Invariant Inference for Distributed Protocols

Weining Cao  
State Key Lab of Novel Software  
Technology, Nanjing University  
Nanjing, China  
weiningcao@smail.nju.edu.cn

Guangyuan Wu  
State Key Lab of Novel Software  
Technology, Nanjing University  
Nanjing, China  
guangyuanwu@smail.nju.edu.cn

Yuan Yao  
State Key Lab of Novel Software  
Technology, Nanjing University  
Nanjing, China  
y.yao@nju.edu.cn

Hengfeng Wei  
Hunan University  
Changsha, China  
hfwei@hnu.edu.cn

Taolue Chen  
Birkbeck, University of London  
London, United Kingdom  
t.chen@bbk.ac.uk

Xiaoxing Ma  
State Key Lab of Novel Software  
Technology, Nanjing University  
Nanjing, China  
xxm@nju.edu.cn

## Abstract

Distributed protocols are notoriously difficult to verify correctly. Proving safety typically requires inductive invariants that both imply the desired property and are preserved by every protocol transition; yet inferring such invariants remains a major bottleneck: existing approaches either restrict the protocol models to a decidable fragment of first-order logic or demand expert-crafted templates.

We present I3DP, a neuro-symbolic framework that synthesizes inductive invariants by executing an IC3-style process over TLA+ states with the assistance of large language models (LLMs). At large, I3DP combines a symbolic IC3 controller which decomposes invariant synthesis into focused blocking tasks, and an LLM which provides protocol-level reasoning that IC3 alone lacks for TLA+ specifications. This integration enables a disciplined yet flexible search for invariants without imposing logical restrictions or requiring manual templates.

We evaluate I3DP on 29 distributed protocols spanning consensus, reconfiguration, and client-server systems, and compare it against Endive, IC3PO, SWISS, DuoAI, and P-FOL-IC3. I3DP discovers candidate invariants for all 29 protocols. Among them, MongoLoglessDynamicRaft models the core mechanism of an industrial-scale Raft-based reconfiguration protocol, and none of the compared tools reports a solution for it. In each case, the invariants synthesized on finite instances are shown in TLAPS to be inductive for the full unbounded protocol, thereby establishing safety.

**CCS Concepts:** • Software and its engineering → Formal software verification.



This work is licensed under a Creative Commons Attribution 4.0 International License.

SOSP '26, Prague, Czech Republic

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2585-2/2026/09

<https://doi.org/10.1145/3830418.3843876>

**Keywords:** distributed protocols, inductive invariants, TLA+, large language models, formal verification

## ACM Reference Format:

Weining Cao, Guangyuan Wu, Yuan Yao, Hengfeng Wei, Taolue Chen, and Xiaoxing Ma. 2026. I3DP: Neuro-Symbolic Inductive Invariant Inference for Distributed Protocols. In *ACM SIGOPS 32nd Symposium on Operating Systems Principles (SOSP '26)*, September 29–October 02, 2026, Prague, Czech Republic. ACM, New York, NY, USA, 16 pages. <https://doi.org/10.1145/3830418.3843876>

## 1 Introduction

Distributed protocols lie at the core of modern infrastructure, but establishing their correctness is notoriously challenging. Safety bugs are often rare, state-dependent, and expensive to reproduce, making even extensive testing unreliable. As a result, protocol designers sometimes adopt formal verification: they specify protocols in formal languages such as TLA+ [20] and establish safety through explicit-state model checking, SMT-based symbolic analysis, or deductive proof [5]. These approaches typically rely on *inductive invariants*: state predicates that (1) hold in every initial state, (2) are preserved by every protocol transition, and (3) imply the target safety property. In practice, however, the main difficulty lies not in checking a given candidate, but in automatically *inferring* one.

Existing automated approaches to inductive invariant synthesis generally involve a trade-off. Some restrict protocol models to decidable fragments of first-order logic (e.g., EPR [24]) to make invariant search tractable. These restrictions, however, exclude common TLA+ modeling patterns and may distort natural protocol structure. Other approaches, such as Endive [28], operate directly on TLA+ specifications but require manually curated predicate pools, thereby shifting much of the synthesis burden back to the users. In practice, discovering inductive invariants remains one of the most labor-intensive steps in the formal verification of distributed protocols, often requiring weeks to months of expert effort [12, 31].

State-of-the-art LLMs demonstrate protocol-level reasoning ability to assist in this process. However, directly asking an LLM to produce a complete inductive invariant remains ineffective: the clauses must satisfy initiation, consecution, and safety implication, and any gap in the conjunction invalidates the entire invariant. What is therefore needed is a *principled decomposition* that reduces this global synthesis task into a sequence of localized, focused subproblems.

IC3 (a.k.a. property-directed reachability, or PDR [1]) is a generic verification method for proving safety properties of transition systems. One key observation is that it provides precisely the kind of decomposition needed to synthesize inductive invariants for distributed protocols. Rather than requiring all protocol-level facts at once, IC3 constructs invariants incrementally: it maintains an explicit sequence of frame over-approximations and progresses by identifying bad states (i.e., states in the frontier frame that can transition to a safety violation). For each such state, IC3 performs a backward search through predecessor states in earlier frames; only when the search reaches a *predecessor-free* state does it learn a *blocking clause*, which is then validated and propagated forward. Each blocking step is local and targeted.

*Our work.* We propose I3DP (Inductive Invariant Inference for Distributed Protocols), a neuro-symbolic framework that executes an IC3-style process over TLA+ states to synthesize inductive invariants with the assistance of LLMs. Our design is based on a principled integration of IC3 and LLMs. The symbolic component of I3DP, referred to as the *IC3 controller*, follows the IC3 procedure, decomposing invariant synthesis into focused blocking tasks that are concrete enough for an LLM to handle. The LLM, as the neural component, complements this process by providing protocol-level reasoning that IC3 alone lacks for TLA+ specifications, while the IC3 framework ensures that every proposed clause is validated and provides a completeness fallback.

More concretely, for each bad state exposed at the frontier frame, the IC3 controller performs a backward search through predecessor states in earlier frames. Once the search reaches a predecessor-free state, the LLM proposes protocol-level blocking clauses that exclude it. Each candidate clause is validated by the verification backend before being added to the current frame and then propagated forward. If no further bad states are found and the LLM fails to produce an admissible clause after repeated attempts, the solver falls back to an exact aggregate blocker that negates all accumulated bad states, preserving finite-instance completeness. The process terminates when two adjacent frames coincide, yielding a candidate inductive invariant. Notably, I3DP does not impose restrictions on the input specification and requires no manually designed templates.

We implement the I3DP design and evaluate it on 29 distributed protocols spanning consensus, leader election, lock services, transaction commit, firewall, and client-server

benchmarks. We compare against Endive [28], IC3PO [9], SWISS [11], DuoAI [35], and P-FOL-IC3 [16]. Remarkably, I3DP successfully discovers candidate invariants for all 29 protocols. One benchmark is MongoLoglessDynamicRaft (MLDR), which models the core mechanism of an industrial-scale Raft-based reconfiguration protocol. None of the five compared tools reports a solution for MLDR. In every solved case, we further prove, in TLAPS [5], that the discovered invariants are inductive for unbounded protocol instances, thereby establishing safety.

*Summary.* Our main contributions include:

- We propose a novel neuro-symbolic integration of IC3’s frame-based decomposition with LLM-generated blocking clauses, turning globally intractable direct invariant synthesis into a sequence of focused per-state blocking tasks that are tractable for LLMs.
- We implement an IC3-based TLA+ tool, I3DP, with an explicit IC3 controller and LLM-generated blocking clauses, requiring neither logical restriction on the input specification nor manual template design.
- We evaluate I3DP on 29 distributed protocols and compare it with existing tools. I3DP discovers proven inductive invariants for all 29 benchmarks, including several that none of the compared tools solve.

*Roadmap.* Section 2 reviews background. Section 3 presents a motivating example. Section 4 and Section 5 present the design and implementation details of I3DP, respectively. Section 6 reports evaluation results. Section 7 discusses case studies and limitations. Section 8 surveys related work. Section 9 concludes the paper and outlines future directions.

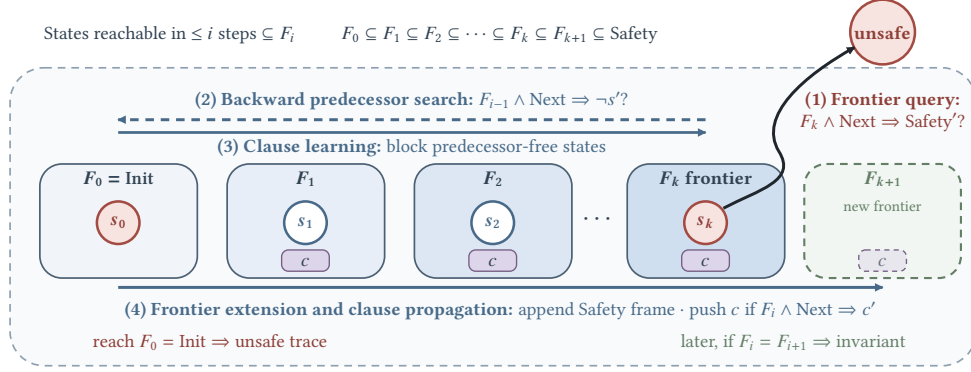
## 2 Background

This section introduces the three foundations that I3DP builds on: the TLA+ specification language (§2.1), the IC3/PDR backward-blocking algorithm (§2.2), and the TLC and Apalache model checkers (§2.3).

### 2.1 TLA+ Specification Language

TLA+ is a high-level specification language for modeling concurrent and distributed systems, based on the Temporal Logic of Actions [20]. It models a system as a transition system over state variables, where each state assigns values to the variables, and each step is described by an action relating the current-state and next-state values.

In practice, a protocol specification typically introduces a tuple of variables *vars*, an initial-state predicate *Init*, and a next-state relation *Next* that captures the allowed transitions. The overall behavior is then written in the standard form  $\text{Init} \wedge [][\text{Next}]_{\text{vars}}$ , meaning that execution starts from a state satisfying *Init* and every step either follows *Next* or leaves the variables unchanged.



**Figure 1.** IC3/PDR backward blocking and clause propagation. Conceptually, frontier processing involves four operations: (1) frontier query, (2) backward predecessor search, (3) clause learning, and (4) frontier extension and clause propagation. The second and third are interleaved during recursive blocking. The process yields either an unsafe trace or an inductive invariant.

For distributed protocols, TLA+ specifications are usually written at the protocol level (rather than the implementation level), using sets, functions, and quantification over nodes, messages, or terms. Safety requirements are expressed as state predicates that must hold in every reachable state. A central verification task is therefore to find an inductive invariant that implies the target safety property, holds initially, and is preserved by *Next*. This is exactly the setting of our work: the protocol is given as a TLA+ transition system, and the challenge is to automatically strengthen the target safety property into an inductive invariant.

## 2.2 IC3/PDR Intuition and Backward Blocking

IC3 [1, 2, 7], also known as property-directed reachability (PDR), is a frame-based algorithm for proving transition-system safety and forms the basis of I3DP.

As shown in Figure 1, IC3 incrementally constructs an inductive invariant using frames that over-approximate bounded-step reachable states:

$$F_0 \subseteq F_1 \subseteq \dots$$

where  $F_0 = \text{Init}$  and  $F_i$  over-approximates the states reachable within  $i$  transitions. Each noninitial frame includes *Safety* (and *Init* must imply *Safety*), and the frames satisfy  $F_i \wedge \text{Next} \Rightarrow F'_{i+1}$ . To *block* a state at level  $i$ , IC3 strengthens  $F_i$  with a clause that excludes the state while preserving all states reachable within  $i$  transitions. IC3 blocks states that can reach a safety violation in finitely many transitions. It repeats this process until it finds an inductive set of states or proves the protocol unsafe. Specifically, IC3 consists of four key operations: (1) frontier query, (2) backward predecessor search, (3) clause learning, and (4) frontier extension and clause propagation.

(1) *Frontier query.* Assume the deepest frame  $F_k$  is the frontier. IC3 first checks whether  $F_k \wedge \text{Next} \Rightarrow \text{Safety}'$ . If the implication does not hold, it yields a *bad state*  $s_k \in F_k$  with

an unsafe successor. IC3 seeks to exclude  $s_k$  and strengthen the frames.

(2) *Backward predecessor search.* During recursive blocking, IC3 interleaves backward predecessor search and clause learning. The former searches backward through the frames for predecessors. For a bad state  $s_i$  at level  $i$ , IC3 checks  $F_{i-1} \wedge \text{Next} \Rightarrow \neg s'_i$ . If the implication does not hold, it yields a predecessor  $s_{i-1}$ , which becomes the next bad-state obligation at level  $i - 1$ , and the search continues toward  $F_0$ . Reaching  $s_0 \in F_0 = \text{Init}$  completes a concrete unsafe trace  $s_0, s_1, \dots, s_k$  and its unsafe successor, proving the protocol unsafe. If the implication holds at level  $i > 0$ ,  $s_i$  is predecessor-free relative to  $F_{i-1}$  and can be blocked at that level.

(3) *Clause learning.* When backward search reaches a state  $s_i$  with no predecessor in  $F_{i-1}$ , IC3 learns a blocking clause  $c$  that is false for  $s_i$  and therefore excludes it from  $F_i$ . Rather than excluding only this state, IC3 seeks a more general clause that can also exclude other states, reducing the number of future blocking obligations. Before admitting  $c$ , IC3 checks two conditions:  $c$  must hold in every initial state, and every transition from a state in  $F_{i-1}$  satisfying  $c$  must lead to another state satisfying  $c$ . The first condition preserves the initial states, while the second makes  $c$  inductive relative to  $F_{i-1}$ . Together, they ensure that  $c$  holds for every state reachable within  $i$  transitions. IC3 can therefore add  $c$  to  $F_1$  through  $F_i$ , strengthening these frames without excluding any state reachable within their respective depths. It then returns to the pending successor state and repeats backward search and clause learning until the entire backward chain is blocked.

(4) *Frontier extension and clause propagation.* Once recursive blocking has discharged all obligations exposed by the frontier query, this operation appends a fresh frame initialized with *Safety*. This advances the frontier, and IC3 propagates admitted clauses into the new frame. A clause  $c$  in  $F_i$  can be pushed to  $F_{i+1}$  when  $F_i \wedge \text{Next} \Rightarrow c'$ . IC3 then repeats the frontier query at the new frontier. If the query fails again,

IC3 recursively blocks the new obligations, strengthening earlier frames as needed. If adjacent frames contain the same clauses, their shared formula contains the initial states, implies Safety, and is closed under Next. The shared formula is therefore an inductive invariant that proves Safety.

### 2.3 TLC and Apalache Model Checkers

TLC [37] is the standard explicit-state model checker for TLA+. It explores the reachable state space of a TLA+ specification by exhaustively enumerating states and transitions, making it particularly effective for detecting safety violations in finite instances of distributed protocols. TLC is widely used in practice due to its simplicity and ability to produce concrete counterexample traces, which are invaluable for debugging protocol designs.

Apalache [17], in contrast, is a symbolic model checker for TLA+ that leverages SMT solving. Instead of enumerating states explicitly, Apalache encodes the transition system and properties into logical constraints and uses an SMT solver to reason about sets of states symbolically. This approach enables more scalable analysis for certain classes of systems and supports bounded model checking, making it well-suited to the witness queries inside our IC3 loop and to exploring behaviors beyond the limits of explicit-state exploration.

TLC and Apalache represent two complementary verification paradigms for TLA+: explicit-state exploration for exhaustive bug finding on finite instances, and symbolic reasoning for more scalable, constraint-based analysis. They are both used in I3DP, as described in §5.

## 3 Motivating Example

We use a two-phase commit protocol modeled in TLA+ to illustrate the challenge of inductive-invariant synthesis and the benefit of IC3 decomposition.

The protocol models  $n$  nodes coordinating a commit-or-abort decision through seven set-valued variables (`vote_yes`, `vote_no`, `alive`, `go_commit`, `go_abort`, `decide_commit`, `decide_abort`) and a boolean `abort_flag`. Nodes vote yes or no; a node that votes no can immediately abort, since the outcome is already determined. A coordinator broadcasts a commit signal (only when all nodes voted yes) or an abort signal (when some node voted no or failed); nodes then finalize their decisions accordingly. Any node may fail at any time. In this model, both voting no and failing set `abort_flag`. The safety property requires: (1) no committed node coexists with an aborted node, (2) every committed node implies all nodes voted yes, and (3) every abort decision carries the abort flag.

**Why is safety not inductive?** Safety alone is not an inductive invariant. Consider the state  $\text{go\_abort} = \{n_1\}$ ,  $\text{abort\_flag} = \text{FALSE}$ , with all other variables at their initial values. This state satisfies Safety (no node has

committed or aborted yet), but from it, node  $n_1$  can execute the Abort action (since  $n_1 \in \text{go\_abort}$ ), producing  $\text{decide\_abort} = \{n_1\}$  with  $\text{abort\_flag} = \text{FALSE}$ , violating the third safety conjunct. An inductive invariant must include additional strengthening clauses that exclude such states.

**Direct LLM synthesis fails.** We test a direct approach: the LLM (Claude Opus 4.6) generates a complete inductive invariant, a model checker verifies the induction condition, and on failure the counterexample is fed back for another attempt. On this two-phase commit protocol, the LLM makes 44 attempts and times out without finding a correct invariant. The failures follow a consistent pattern. Each candidate is plausible, with 13–26 clauses that are largely individually correct (e.g.,  $\text{vote\_yes} \cap \text{vote\_no} = \{\}$ ), but each also contains a few overly strong clauses that exclude reachable states, making the conjunction fail the induction check. For instance, attempts frequently assert  $\text{go\_abort} \neq \{\} \Rightarrow \text{vote\_yes} \neq \text{Node}$  (an abort signal implies not all nodes voted yes). This is too strong: since crashed nodes do not recover in the simplified model, a node may vote yes and then crash, and the coordinator’s resulting abort broadcast reaches a state where `go_abort` is nonempty yet every node voted yes. Crucially, the induction counterexample refutes only the full conjunction and never identifies the offending clause, making the LLM oscillate between variants without converging.

A natural remedy is to give the LLM finer-grained feedback. We extend the loop with clause-level feedback from TLC, which identifies the specific clause that the counterexample violates. This does not help either: the feedback steers the LLM toward locally repairing the offending clause rather than reconsidering the invariant as a whole, and direct synthesis still fails to converge on this protocol. The problem is not specific to this example. Across our benchmark suite, this clause-level feedback strategy even causes one additional protocol, `tlapl-simpleregular`, to fail although it otherwise succeeds without such feedback.

**IC3 decomposition.** The failure suggests an alternative strategy: decompose the synthesis task so that each step targets a single concrete bad state. Interestingly, IC3’s frame-based structure offers exactly this decomposition: clauses are organized into successive frames; each frontier query exposes one bad state, and the LLM only needs to propose clauses that block that state.

We apply this IC3-based approach to the same protocol. The IC3 controller queries the frontier frame and finds a bad state:  $\text{go\_abort} = \{n_1\}$ ,  $\text{alive} = \{n_1\}$ , with all other variables at their initial values. Backward predecessor search confirms it is predecessor-free. The LLM then produces 15 candidates targeting this state; the controller selects  $\text{go\_abort} = \{\} \vee \text{go\_abort} = \text{Node}$ , validates it, and admits it to the current frame. The next frontier query reveals a new bad state, which is already covered by an existing clause in the

pool ( $\text{go\_commit} = \{\} \vee \text{go\_commit} = \text{Node}$ ), and thus no additional LLM call is needed. This process continues for 7 frontier rounds, of which only 2 require new LLM calls, with the remaining 5 reusing previously learned clauses. After frontier blocking and clause pushing, during which all push failures are resolved by reusing previously learned clauses without additional LLM calls, adjacent frames converge, resulting in a fixpoint of 11 strengthening clauses, automatically minimized to 8, obtained in 171 seconds with a total of 2 LLM calls.

Importantly, the IC3 controller, rather than the LLM, decides which clauses to keep. Candidate clauses are filtered through admission checks, and only validated clauses are propagated forward. Incorrect or overly strong clauses are discarded during admission (rather than silently corrupting the invariant), so that IC3 compensates for LLM imprecision by construction. Beyond mitigating LLM imprecision, the IC3 structure also provides a finite-instance completeness backstop. Namely, if no admissible LLM-guided clause can block the remaining bad states and no further bad states are found by Apalache, the solver falls back to an exact-checked aggregate blocker that excludes all existing bad states. In contrast, flat CTI-elimination approaches such as Endive [28] do not provide this completeness guarantee.

**Takeaway.** Direct synthesis requires the LLM to get all clauses correctly at once, and an induction failure over the full conjunction provides little guidance on which clause is failing. The IC3 decomposition circumvents the problem: each frontier query exposes a concrete bad state, the LLM proposes clauses to block that state, and the verification backend validates each clause individually before admission. This localisation of reasoning and feedback makes the search both more tractable and more robust, which underpins the design of I3DP.

## 4 Design

Figure 2 presents an overview of I3DP, which consists of three components: the **IC3 controller** that maintains frames and manages backward search from frontier bad states, the **verification backend** that handles property-directed witness queries and screens candidate blocking clauses against concrete reachable states, and the **LLM module** that proposes candidate blocking clauses from protocol context and representative failing states.

### 4.1 IC3 Controller

The IC3 controller maintains the explicit frame sequence  $\text{Frames} = [\{\text{Init}\}, \{\text{Safety}\}]$ , where  $F_0 = \text{Init}$  and each frame  $F_i$  with  $i > 0$  begins from an internal Safety base that is then strengthened by admitted blocking clauses. Each iteration proceeds through four stages.

**Stage 1: Frontier Query.** The controller queries the current deepest frame for a *frontier bad state*, i.e., a state that can lead

to a safety violation in one transition. If no such state exists, the frontier is already safe in one step and the controller proceeds to the push phase. Otherwise, the bad state enters the backward predecessor search.

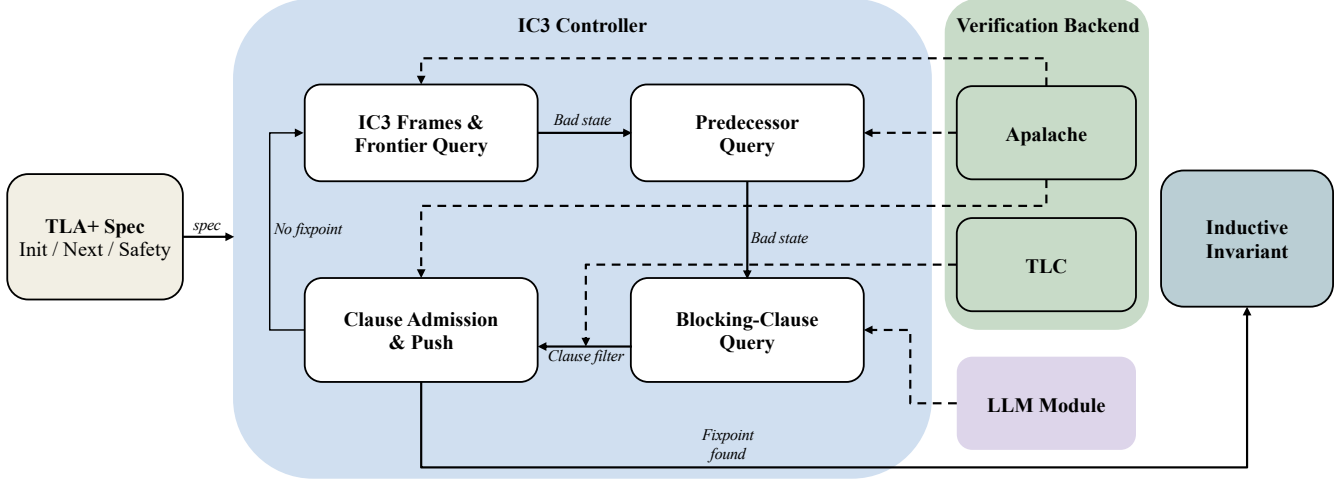
**Stage 2: Predecessor Query.** The controller maintains a worklist of bad states, each paired with its frame level. For each bad state, it issues a predecessor query one level shallower each time: if a predecessor exists, the bad state is replaced by its predecessor at the lower frame level and the search descends further; if no predecessor exists at a level above zero, the bad state is *predecessor-free* and triggers blocking-clause search. If the search reaches frame level 0, a real counterexample exists and the controller reports failure.

**Stage 3: Blocking-Clause Query.** Once backward predecessor search reaches a predecessor-free bad state, the controller first consults previously learned clauses (i.e., a pool of all blocking clauses retained across the full IC3 process) for any that already block it. Note that standard IC3 does not need such a pool because clause derivation via SAT generalization is deterministic. However, with LLM-generated clauses, a clause originally learned for one bad state may later block a different bad state at a different frame level.

If existing clauses suffice, they are inserted into the relevant frames and the covered bad state is removed from the worklist without invoking the LLM. Otherwise, the controller invokes the LLM module to propose candidate blocking clauses from the current batch of bad states. Returned candidates are screened against concrete reachable states and retained for later reuse when they pass. Clauses enter IC3 frames only after satisfying the initial-state implication and the target-frame admission condition, confirming that each clause is inductive relative to the frame’s current contents.

If blocking fails, the bad state remains in the worklist and the controller re-queries the frontier, conjoining the negation of all known bad states, to obtain a fresh frontier bad state and try again. If no new frontier bad states can be produced (meaning the known bad states already suffice to block the frontier), the controller negates all known bad states and adds the resulting negation to the frame as a last-resort blocking clause after exhausting LLM-guided clause search, preserving the finite-instance completeness backstop of IC3. This guarantee distinguishes I3DP from flat CTI-elimination approaches: even when the LLM fails to propose a useful clause, the IC3 loop still converges on the checked finite instance.

**Stage 4: Clause Push.** Once the frontier is fully blocked, the controller appends a fresh frame and enters the push phase: each admitted clause is propagated toward deeper frames as far as the IC3 induction check allows. A clause that cannot be pushed is simply non-inductive for the next frame; the solver makes one opportunistic blocking attempt on the push-failure counterexample, first consulting retained clauses and then issuing at most one additional LLM query



**Figure 2.** Overview of I3DP. The IC3 loop iterates through four stages: frontier query, backward predecessor search, blocking-clause search, and clause push. They are coordinated by three components: (1) an IC3 controller that maintains frames and manages backward search from frontier bad states, (2) a verification backend where Apalache handles symbolic witness queries and TLC screens candidate blocking clauses against concrete reachable states, and (3) an LLM module that proposes candidate blocking clauses from protocol context and representative bad states.

if needed; if no admissible clause is found, it moves on. After propagation, the controller checks whether any two adjacent frames have become equal. Equal adjacent frames constitute a fixpoint: the shared frame formula is extracted as the candidate invariant.

**Key deviations from Standard IC3.** Our design builds on IC3, which effectively transforms the global invariant synthesis problem into a sequence of localized, per-state blocking tasks. This structure aligns naturally with LLM-based clause generation, where each query can focus on a concrete sub-problem rather than the full invariant. However, I3DP also features key deviations from standard IC3.

The main adaptation lies in how blocking clauses are learned. In standard IC3, every predecessor-free bad state is blocked immediately by a SAT-derived Boolean cube generalization, a mechanical step that always succeeds. In contrast, TLA+ specifications of distributed protocols involve sets, functions, and quantifiers that have no natural Boolean-cube representation, and a single bad state may not give the LLM sufficient context to infer the appropriate protocol-level condition. To address this, I3DP allows blocking to be deferred: when no admissible clause is found for a given bad state, the controller continues exploring the frontier, accumulating additional bad states in a shared worklist. The resulting batch is then presented to the LLM, providing a richer context for generalization across multiple related failures rather than handling bad states in isolation.

Moreover, I3DP adds opportunistic blocking during the push phase. When a clause fails to propagate to the next frame, the resulting push-failure counterexample exposes a

bad state at the source frame. Standard IC3 simply leaves the clause in its current frame, but I3DP performs an additional blocking attempt: it first checks whether existing clauses suffice, and otherwise queries the LLM for a new clause targeting this push-failure state. Any admissible clause is validated and incorporated under the same checks to guarantee soundness. If no such clause is found, the solver simply proceeds as usual, treating the push failure as a normal non-inductiveness event.

## 4.2 Verification Backend and LLM Module

I3DP combines two model checkers as the verification backend, together with an LLM module, to support the IC3 controller.

**Symbolic witness queries (Apalache).** The frontier and predecessor queries inside the IC3 loop are satisfiability problems: “does there exist a state in the current frame that can step to a safety violation?” and “does there exist a predecessor of this bad state in the shallower frame?” We hence use Apalache (cf. §2.3), whose SMT-based encoding handles quantified TLA+ state spaces without explicit enumeration. This is essential because the query spaces of frontier and predecessor witnesses can exceed the limits of explicit-state enumeration for some protocols.

**Concrete clause screening (TLC).** Once the LLM proposes candidate blocking clauses, they must be validated against the protocol’s reachable behavior. We use TLC (cf. §2.3) to screen candidates in batch on the configured finite instance by checking them against concrete reachable states. This is a model-checking problem over a known, finite state space,

where TLC’s explicit enumeration is particularly effective. Clauses which are falsified by reachable states are discarded early, and the corresponding counterexamples provide concrete diagnostic feedback to the LLM, steering subsequent proposals away from the same overgeneralization. Moreover, TLC supports clause prioritization: for each valid clause, it counts how many current bad states are excluded and uses this metric to rank clauses for admission into the frame.

**LLM clause generation.** The LLM module receives the full TLA+ specification along with the current batch of bad states, and proposes candidate blocking clauses expressed as quantified TLA+ predicates over protocol-level variables. Unlike SAT-derived Boolean cubes that flip specific state assignments, these clauses capture protocol-level invariant conditions, such as “every node in the quorum has voted” or “no committed log entry can be overwritten by a lower term”, allowing them to block entire families of bad states at once and accelerate convergence.

A key design choice is to query the LLM for individual blocking clauses scoped to specific bad states at a given frame level, rather than asking for a complete invariant. This crucial decomposition creates a focused, tractable subproblem: the LLM needs only to propose predicates that exclude the given bad states while preserving reachable behavior, rather than discovering all clauses simultaneously. Batching multiple bad states into a single call further strengthens generalization, enabling the LLM to detect common structural patterns and to produce clauses that rule out families of violations rather than individual instances.

The LLM also receives accumulated verification feedback (e.g., which prior clauses held on reachable states, which failed, and the specific falsifying states), steering it away from repeated overgeneralization and creating a productive verification-generation feedback loop.

## 5 Implementation

This section presents the implementation details of I3DP, including the IC3 controller algorithm, the selection of verification backends and their respective roles, and the design of LLM prompts and response handling for blocking clause generation.

### 5.1 IC3 Main Loop

Algorithm 1 presents the complete IC3 control flow (corresponding to the IC3 controller highlighted in blue in Figure 2). The controller initializes  $Frames = [\{Init\}, \{Safety\}]$  and maintains a global  $clauseMemory$  that retains every filtered blocking clause across the full IC3 process, allowing later iterations to reuse clauses discovered during earlier frontier-blocking phases without re-querying the LLM. Each iteration proceeds in three steps.

---

#### Algorithm 1: IC3 Loop with LLM-Generated Blocking Clauses

---

**Input:** Initial condition  $Init$ , safety property  $Safety$   
**Output:** Invariant  $Inv$ , or **UNSAFE** if the protocol is unsafe

```

1  $F \leftarrow [\{Init\}, \{Safety\}];$ 
2  $clauseMemory \leftarrow \emptyset;$ 
3 while true do
4   if not  $BLOCKFRONTIER(F, clauseMemory)$  then
5     return UNSAFE;
6   end
7    $PUSHClausesForward(F);$ 
8    $j \leftarrow FINDFIXPOINT(F);$ 
9   if  $j \neq nil$  then
10     $Inv \leftarrow EXTRACTINVARIANT(F_j);$ 
11    return  $Inv;$ 
12  end
13 end
```

---

First, the controller runs  $BLOCKFRONTIER$  on the current deepest frame. If backward search reaches frame level 0, it returns **false** and the controller reports **UNSAFE**.

Second, once  $BLOCKFRONTIER$  has blocked all bad states reachable in one step from the current frontier frame,  $PUSHClausesForward$  appends a fresh frame and propagates each admitted clause toward deeper frames by checking  $F_k \wedge Next \Rightarrow c'$ ; clauses that pass are pushed to the next frame. An unpushable clause produces a push-failure pre-state, which triggers one opportunistic blocking attempt against  $clauseMemory$  and, if needed, at most one LLM query, subject to the standard clause-admission check ( $Init \Rightarrow c$  and  $F_{k-1} \wedge c \wedge Next \Rightarrow c'$ ); if no admissible clause is found, the solver simply moves on and the pre-state is discarded. Any new clause added to a frame triggers re-pushing from that frame.

Third,  $FINDFIXPOINT$  scans adjacent frame pairs and returns the index  $j$  of the first pair satisfying  $F_j = F_{j+1}$ ; if found, the controller extracts and returns the candidate invariant.

### 5.2 Frontier Frame Bad-State Blocking

Algorithm 2 details the frontier-blocking phase. It queries the current deepest frame for a state from which one transition reaches a Safety-violating successor and adds this pre-state to the bad-state worklist  $badStates$ . A predecessor query then checks whether  $F_{k-1} \wedge Next \Rightarrow \neg badState'$ ; if a predecessor is found, the bad state is replaced by its predecessor at the lower frame level, and the search descends further. If the search reaches frame level 0, a real counterexample path to a safety violation exists and the procedure returns **false**. If no predecessor exists at any frame level above zero, the bad state is *predecessor-free* and triggers blocking-clause search. Predecessor checks are performed per bad state.

Before invoking the LLM, the loop first calls  $ADMITCLauses$  on the current clause memory; if an existing clause

---

**Algorithm 2: BLOCKFRONTIER: Frontier Bad-State Blocking**

---

**Input:** Frame sequence  $F$ , clause memory  $clauseMemory$ **Output:** **true** if no more bad states

```
1  $badState \leftarrow \text{FRONTIERQUERY}(F);$ 
2  $badStates \leftarrow \{badState\};$ 
3 while  $badStates \neq \emptyset$  do
4    $(badState, frameLevel) \leftarrow$ 
      $\text{PREDECESSORQUERY}(badState, F);$ 
5    $badStates \leftarrow badStates \cup \{badState\};$ 
6   if  $frameLevel = 0$  then
7     return false
8   end
9    $clauses \leftarrow \text{ADMITCLAUSES}(clauseMemory, badStates);$ 
10  if  $clauses \neq \emptyset$  then
11     $badStates \leftarrow \text{REMOVEBLOCKED}(badStates, clauses);$ 
12    continue;
13  end
14   $candidateClauses \leftarrow \text{LEARNCLAUSES}(badStates, F);$ 
15   $candidateClauses \leftarrow \text{FILTER}(candidateClauses, F);$ 
16   $clauseMemory \leftarrow clauseMemory \cup candidateClauses;$ 
17   $clauses \leftarrow \text{ADMITCLAUSES}(clauseMemory, badStates);$ 
18   $badStates \leftarrow \text{REMOVEBLOCKED}(badStates, clauses);$ 
19   $badState \leftarrow \text{FRONTIERQUERY}(F, badStates);$ 
20   $badStates \leftarrow badStates \cup \{badState\};$ 
21 end
22 return true;
```

---

can cover the current bad states and is admissible, the loop continues without querying the LLM. Only when no retained clause is admissible does the loop call `LEARNCLAUSES`, which submits the protocol specification, the current batch of bad states, verification histories of prior LLM-generated clauses, and existing clauses in the current frame to the LLM. If this still does not yield an admissible clause, the bad state remains in the worklist, and the next iteration attempts to discover additional bad states and generate new clauses.

New candidate clauses are filtered via `FILTER` and merged into  $clauseMemory$ : TLC checks each candidate against the accumulated reachable states and discards those that are falsified, since they cannot appear in the final invariant and would only waste subsequent verification attempts (cf. §5.3).

Moreover, TLC checks clauses in  $clauseMemory$  against the current bad states and retains only those that block at least one bad state; among the retained clauses, those covering more bad states are preferred, and when coverage is equal, shorter clauses are preferred. `ADMITCLAUSES` is then called again on the updated memory. The admission check verifies  $Init \Rightarrow c$  and  $F_{k-1} \wedge c \wedge Next \Rightarrow c'$  for each candidate  $c$ , inserting admitted clauses into frames from level 1 up to the current level. Once any clause is successfully admitted, the current blocking attempt for this bad state terminates. After admission, `REMOVEBLOCKED` removes from

$badStates$  every bad state covered by the newly admitted clauses; `FRONTIERQUERY` then re-queries the frontier, excluding already-seen pre-states, so that newly exposed frontier bad states re-enter the same round.

### 5.3 Verification Backend

As stated in § 4.2, Apache and TLC serve complementary roles in I3DP.

Apache handles all symbolic queries in the IC3 loop: the frontier query (lines 1 and 19 in Algorithm 2), the predecessor query (line 4), clause admission ( $Init \Rightarrow c$  and  $F_{k-1} \wedge c \wedge Next \Rightarrow c'$ ), and clause push ( $F_k \wedge Next \Rightarrow c'$ ). All rely on its symbolic one-step reachability encoding, which avoids state enumeration and returns concrete witness states directly.

TLC screens candidate clauses in batches: the entire candidate pool is validated against reachable states in a single model-checking pass rather than checking each clause individually. For each surviving clause, TLC counts how many current bad states it excludes; this coverage count ranks candidates before admission.

Apache is chosen for witness queries because the search spaces of frontier and predecessor queries can exceed the limits of explicit-state enumeration for some protocols, whereas TLC's batched explicit-state checking is more efficient for screening concrete clause candidates.

### 5.4 LLM Clause Generator

Each prompt includes (1) the TLA+ protocol specification, (2) a batch of current bad states, (3) quantifier templates extracted from the specification, (4) the persistent cross-round clause memory (whose clauses should not be repeated), and (5) per-clause observations tracking which candidates have previously held or failed on reachable states. The prompt requests complete TLA+ clauses as a JSON batch response. Prompt 1 is an example of one of our concrete templates. It has been abridged for presentation.

The LLM response is parsed as a JSON array, from which individual clauses are extracted, normalized, and syntax-checked with SANY [20]. The clauses are then filtered with TLC against reachable states. Together, SANY syntax checking and TLC-based screening serve as additional filters, eliminating malformed or already-falsified candidates early.

Note that common scoping mistakes, such as reusing a model constant as a quantifier-bound variable, are automatically detected and repaired during normalization. Quantifier templates are extracted directly from the specification and deduplicated modulo variable renaming, providing the LLM with protocol-specific quantifier structure without requiring manual template design.

## 6 Evaluation

This section presents an empirical evaluation of I3DP with emphasis on the following research questions (RQs).

### Prompt 1: Blocking-Clause Generation Prompt (Abridged for Presentation)

Generate TLA+ inductive invariant clauses that exclude the counterexample states while capturing structural relationships in the specification.

1. Complete, self-contained TLA+ expressions; no primed variables or temporal operators.
2. Use nested quantifiers for dependent domains, reusing the provided quantifier templates.
3. Generalize from counterexamples; prefer structural clauses over witness-specific exclusions.
4. Use only identifiers and operators present in the specification or counterexample states.
5. Do not repeat existing clauses or return any clause listed under known false.
6. Use known true clauses as structural guidance only; try strengthening them into tighter variants.
7. For each known false entry, try loosening the clause by folding the violating state into its quantified or value space.

**Specification:** {spec\_content}

**Counterexample states:** {bad\_states}

**Quantifier templates:** {quantifier\_templates}

**Existing clauses:** {existing\_clauses}

**Known true on finite reachable states:**

{known\_true\_clauses}

**Known false on finite reachable states:**

{known\_false\_clauses}

**Output:** JSON array of 10–15 clauses: [{"clause\_name": "...", "clause": "TLA+ expr"}]

- RQ1.** How effective is I3DP in inductive-invariant synthesis, and how does it compare with state-of-the-art tools?
- RQ2.** What is the contribution of the IC3 architecture to I3DP’s effectiveness?

## 6.1 Benchmark and Setup

We evaluate I3DP on 29 distributed protocols spanning consensus, leader election, lock services, transaction commit variants [10], firewalls, client–server protocols, Paxos variants [18, 19], and a larger production-style case study (MongoLoglessDynamicRaft). The suite draws primarily from the Endive benchmark collection and extends it with an additional hard Paxos variant sourced from the IC3PO repository and adapted to TLA+. Endive is directly comparable on the 28 shared TLA+ protocols; IC3PO [9], SWISS [11], DuoAI [35], and P-FOL-IC3 [16] serve as additional baselines wherever a matching benchmark is available in their respective input languages. We do not include Kondo [38] and Basilisk [39] as baselines for two reasons. First, they are built on Dafny, and translating specifications from TLA+ to Dafny is non-trivial. For example, it can require decomposing atomic multi-host

actions into per-host steps. Second, the invariants generated by these two tools do not directly include the safety property, so proving the safety property may require manual effort such as supplying hints and writing implication proofs.

We impose a 600-second timeout per tool on all protocols. For pyv-paxos-epr and mldr, we extend the limit to 2 hours for all tools because the verification burden for these two protocols is substantially heavier.

**Verification cost.** Inductiveness checking can itself be expensive. On mldr, a single inductiveness check by I3DP can take more than five minutes, so verification alone can consume a substantial fraction of the default timeout. This cost motivates the extended timeout for the two hardest protocols.

We note that Ivy-based tools (IC3PO [9], DuoAI [35], SWISS [11], and P-FOL-IC3 [16]) verify invariants for unbounded instances directly. In contrast, TLA+-based tools (I3DP and Endive) discover invariants on finite instances and rely on a separate TLAPS proof for unbounded correctness. However, TLA+ supports a broader class of protocol features than Ivy. For mldr, Ivy-based tools cannot prove the invariant discovered by I3DP for unbounded instances, as the protocol lies outside Ivy’s decidable fragment. Moreover, even under the same finite-instance size used by I3DP, they fail to propose an invariant that passes the inductiveness check. We therefore report them as TO in Table 1, which can also be understood as out of scope for these tools. Finite instance sizes we use for I3DP follow Endive’s configuration and are set manually.

We report end-to-end wall-clock runtime for each protocol, all measured on the same machine: a MacBook Pro with an Apple M4 Pro chip (12 cores: 8 performance and 4 efficiency) and 48 GB of memory.

**Run-to-run stability.** Although individual LLM responses are nondeterministic, I3DP behaves consistently across repeated runs. TLC filters out invalid clauses, and multiple independent LLM calls in each run reduce sensitivity to any single response. Consequently, variations in individual LLM outputs do not materially affect whether I3DP discovers an inductive invariant.

## 6.2 RQ1: Effectiveness and Comparison

Table 1 provides a full per-protocol breakdown. We compare I3DP against five prior state-of-the-art automated invariant inference tools: Endive [28], IC3PO [9], SWISS [11], DuoAI [35], and P-FOL-IC3 [16]. Because Endive requires expert-curated quantifier and predicate templates, we do not report Endive results on the new benchmark protocol (shown as N/A in Table 1).

**Baseline tools.** Endive [28] is the first invariant inference tool to operate directly on TLA+ specifications, using a flat CTI-elimination loop with a manually curated predicate pool; this design requires protocol-specific insight to

craft the quantifier and predicate pool, making it hard for non-experts to apply. IC3PO [9] extends IC3/PDR with symmetric incremental induction over indistinguishable domain sorts, operating on Ivy specifications in decidable first-order fragments; it is highly effective when protocol correctness aligns with strong structural regularity but less general for invariants involving ordering, arithmetic, or non-symmetric relational structure. SWISS [11] performs symmetry-aware exhaustive search over template-defined candidate invariant spaces with counterexample-based pruning. DuoAI [35] extends DistAI [36] with support for existential quantification, using data-driven enumeration combined with SMT-based refinement to infer invariants with alternating quantifiers. P-FOL-IC3 [16] extends IC3 to infer invariants with quantifier alternations on mypyvy specifications, using a breadth-first search strategy to tame the search space explosion; it is one of the few tools capable of handling quantifier alternations but is limited to mypyvy’s decidable fragment.

**Effectiveness.** Across the 29 protocols in Table 1, I3DP solves all 29. The hardest case is `mldr` (MongoLoglessDynamicRaft), which takes 4525 seconds. For all 29 solved benchmarks, TLAPS confirms that the discovered invariants are inductive for unbounded protocol instances, thereby establishing the corresponding safety arguments.

**Comparison with Endive.** I3DP eliminates Endive’s reliance on manual predicate-pool design by replacing predicate enumeration with LLM-based clause generation guided by concrete counterexample states. This design makes I3DP fully automated on new specifications without requiring protocol-specific templates.

On the 28 shared TLA+ protocols, I3DP decisively outperforms Endive, solving 28 while Endive solves 24. Beyond the automation gap, Endive’s flat CTI-elimination loop lacks an IC3-style completeness backstop. By retaining the IC3 controller, I3DP preserves the finite-instance completeness guarantee while shifting clause discovery to LLM-generated blocking clauses.

**Comparison with IC3PO, SWISS, DuoAI, and P-FOL-IC3.** IC3PO solves 26 of the 29 benchmarks and is often faster than I3DP on the protocols it covers, but it times out on the three hardest cases (`pyv-learning-switch`, `mldr`, and `pyv-paxos-epr`). As discussed above, its symmetry-based method is less general for protocols requiring richer ordering or arithmetic reasoning. SWISS solves 17 of the 29 benchmarks, timing out on 8 and terminating without an invariant on 4. DuoAI solves 21 of the 29 benchmarks, succeeding on many protocols where its predecessor DistAI struggled, but it inherits Ivy’s logical restrictions and cannot handle protocols requiring ordering, arithmetic, or non-symmetric relational structure. P-FOL-IC3 solves 24 of the 29 benchmarks and is fast on simpler protocols, but it times out on five harder cases including `i4-learning-switch`, `pyv-consensus-forall`,

`pyv-learning-switch`, `pyv-paxos-epr`, and `mldr`. It is restricted to mypyvy’s decidable fragment and, like IC3PO, does not extend to protocols requiring rich ordering or arithmetic.

### 6.3 RQ2: Contribution of the IC3 Architecture

To isolate the contribution of the IC3 loop, we evaluate two ablation variants: (1) LLM-alone, which uses the same underlying model (Claude Opus 4.6) but omits the IC3 frame structure entirely, and (2) LLM-var, which replaces Claude Opus 4.6 with Gemini 3 Flash inside the full I3DP.

**LLM-alone baseline.** Given the TLA+ specification and the safety property, the baseline prompts the LLM to propose a complete inductive invariant, i.e., a single conjunctive formula that must explicitly include `TypeOK` and the safety property. The candidate invariant is then checked for syntactic correctness as well as proof obligations, i.e.,  $\text{Init} \Rightarrow \text{Inv}$  and  $\text{Inv} \wedge \text{Next} \Rightarrow \text{Inv}'$ . If the candidate fails due to a syntax error, violation of the initialization condition, or lack of inductiveness, the failure is reported back to the LLM, and a new candidate is requested. This process iterates until a valid candidate is found or a per-benchmark time budget is exhausted (600 s by default; 2 h for `mldr` and `pyv-paxos-epr`).

On the 29 protocols, LLM-alone solves 24 instances and times out on 5 (`tla-twophase`, `i4-two-phase-commit`, `pyv-learning-switch`, `mldr`, and `pyv-paxos-epr`). On `mldr`, the baseline gets stuck: it makes 62 clause-generation attempts within 10 minutes yet keeps proposing variants of the same insufficient clauses without making progress. An extended run exceeding 24 hours likewise fails to escape this loop and never produces the key four-quantifier clause. In contrast, I3DP continuously accumulates new bad states through the IC3 loop, changing the context available to later LLM calls and eventually guiding the search toward the required global properties. These results indicate that an LLM alone can solve many of the simpler benchmarks, but the harder instances require the IC3 structure to effectively decompose the invariant inference task.

**Weaker LLM with IC3.** The last column of Table 1 reports the result where the default Claude Opus 4.6 is replaced with Gemini 3 Flash. Under this substitution, I3DP solves 27 of the 29 benchmarks, timing out on `mldr` and `pyv-paxos-epr`.

Notably, the LLM-alone baseline using Claude Opus 4.6 fails to solve `tla-twophase`, whereas the weaker Gemini 3 Flash paired with I3DP’s IC3 controller succeeds on the same protocol. This comparison highlights that the IC3 decomposition can compensate for limitations of the underlying model: the frame structure, clause admission, and backward predecessor search provide enough guidance for even a weaker model to produce useful blocking clauses, when each query is focused on a single concrete bad state.

**Table 1.** Comparison with the default I3DP configuration (Claude Opus 4.6), two ablation variants (LLM-alone, LLM-var), and five prior automated invariant inference tools (Endive, IC3PO, SWISS, DuoAI, P-FOL-IC3). “Inv” reports the number of clauses in the discovered invariant (excluding TypeOK, including the safety-property clauses), which partially reflects the complexity of subsequent proof writing. “TO” = timeout, “ERR” = tool error, “FAIL” = terminated without finding an invariant, “N/A” = no comparable result. Time is reported in seconds.

| # Protocol                        | I3DP |     | Endive |     | IC3PO |     | P-FOL-IC3 |     | SWISS |     | DuoAI |     | LLM-alone |     | LLM-var |     |
|-----------------------------------|------|-----|--------|-----|-------|-----|-----------|-----|-------|-----|-------|-----|-----------|-----|---------|-----|
|                                   | Time | Inv | Time   | Inv | Time  | Inv | Time      | Inv | Time  | Inv | Time  | Inv | Time      | Inv | Time    | Inv |
| 1 tla-consensus                   | 1    | 2   | 0      | 2   | 0     | 1   | 0         | 1   | 0     | 3   | 1     | 2   | 13        | 3   | 2       | 2   |
| 2 tla-tcommit                     | 1    | 1   | 1      | 1   | 0     | 2   | 1         | 2   | FAIL  | 1   | 2     | 7   | 12        | 4   | 2       | 1   |
| 3 i4-lock-server                  | 60   | 2   | 3      | 2   | 0     | 2   | 0         | 2   | 0     | 2   | 1     | 2   | 9         | 3   | 32      | 2   |
| 4 ex-quorum-leader-election       | 41   | 2   | 6      | 2   | 1     | 4   | 5         | 7   | 32    | 6   | 2     | 9   | 13        | 7   | 56      | 2   |
| 5 pyv-toy-consensus-forall        | 42   | 3   | 8      | 3   | 2     | 4   | 1         | 6   | 13    | 5   | 2     | 6   | 10        | 6   | 68      | 3   |
| 6 tla-simple                      | 50   | 2   | 4      | 2   | 2     | 4   | 0         | 2   | 25    | 8   | 247   | 327 | 15        | 7   | 52      | 3   |
| 7 ex-lockserv-automaton           | 92   | 9   | 9      | 9   | 2     | 9   | 5         | 10  | FAIL  | 1   | 2     | 13  | 12        | 12  | 222     | 9   |
| 8 tla-simpleregular               | 45   | 3   | 4      | 4   | 1     | 4   | 6         | 4   | 31    | 21  | TO    |     | 331       | 7   | 59      | 2   |
| 9 pyv-sharded-kv                  | 117  | 6   | 132    | 6   | 1     | 6   | 13        | 10  | TO    |     | 2     | 13  | 56        | 10  | 110     | 5   |
| 10 pyv-lockserv                   | 90   | 9   | 14     | 9   | 2     | 9   | 5         | 8   | TO    |     | 1     | 13  | 106       | 10  | 303     | 8   |
| 11 tla-twophase                   | 165  | 9   | 19     | 10  | 4     | 11  | 4         | 9   | FAIL  | 1   | 2     | 41  | TO        |     | 318     | 9   |
| 12 i4-learning-switch             | 434  | 12  | TO     |     | 11    | 9   | TO        |     | 551   | 13  | 10    | 20  | 16        | 14  | 462     | 10  |
| 13 ex-simple-decentralized-lock   | 35   | 3   | 17     | 4   | 3     | 4   | 0         | 3   | 7     | 3   | 6     | 18  | 24        | 3   | 43      | 3   |
| 14 i4-two-phase-commit            | 171  | 11  | 30     | 13  | 2     | 11  | 10        | 10  | 6     | 14  | 1     | 21  | TO        |     | 209     | 11  |
| 15 pyv-consensus-wo-decide        | 92   | 5   | 52     | 8   | 75    | 6   | 26        | 9   | 38    | 8   | 3     | 23  | 15        | 9   | 243     | 7   |
| 16 pyv-consensus-forall           | 383  | 6   | 96     | 8   | 377   | 6   | TO        |     | 36    | 11  | 9     | 27  | 11        | 9   | 198     | 6   |
| 17 pyv-learning-switch            | 315  | 6   | TO     |     | TO    |     | TO        |     | TO    |     | 13    | 32  | TO        |     | 227     | 7   |
| 18 pyv-sharded-kv-no-lost-keys    | 188  | 6   | 5      | 2   | 0     | 2   | 1         | 2   | 2     | 4   | 2     | 13  | 10        | 3   | 101     | 6   |
| 19 ex-naive-consensus             | 55   | 4   | 15     | 4   | 2     | 4   | 4         | 8   | TO    |     | ERR   |     | 16        | 7   | 129     | 4   |
| 20 pyv-client-server-ae           | 153  | 2   | 19     | 2   | 0     | 2   | 0         | 2   | 17    | 5   | 2     | 3   | 9         | 3   | 36      | 2   |
| 21 ex-simple-election             | 46   | 3   | 10     | 4   | 1     | 4   | 1         | 3   | TO    |     | ERR   |     | 10        | 5   | 147     | 4   |
| 22 pyv-toy-consensus-epr          | 36   | 3   | 10     | 4   | 2     | 4   | 2         | 6   | 10    | 5   | 3     | 4   | 9         | 6   | 80      | 4   |
| 23 ex-toy-consensus               | 34   | 2   | 4      | 2   | 3     | 3   | 5         | 5   | TO    |     | ERR   |     | 11        | 4   | 44      | 2   |
| 24 pyv-client-server-db-ae        | 90   | 4   | TO     |     | 23    | 7   | 6         | 8   | 60    | 15  | 3     | 5   | 19        | 9   | 178     | 5   |
| 25 pyv-firewall                   | 35   | 2   | 24     | 5   | 2     | 3   | 0         | 2   | FAIL  | 1   | ERR   |     | 12        | 3   | 68      | 2   |
| 26 ex-majorityset-leader-election | 40   | 5   | 27     | 5   | 14    | 6   | 75        | 16  | 24    | 11  | ERR   |     | 23        | 9   | 62      | 5   |
| 27 pyv-consensus-epr              | 122  | 6   | 207    | 8   | 68    | 6   | 569       | 21  | 26    | 8   | 4     | 14  | 11        | 9   | 134     | 6   |
| 28 mldr                           | 4525 | 16  | TO     |     | TO    |     | TO        |     | TO    |     | TO    |     | TO        |     | TO      |     |
| 29 pyv-paxos-epr                  | 1806 | 9   | N/A    |     | TO    |     | TO        |     | TO    |     | ERR   |     | TO        |     | TO      |     |

**Summary.** The ablation studies demonstrate that the IC3 architecture is one of the key contributors to I3DP’s effectiveness. The frame-based decomposition and clause admission checks mitigate variability in clause quality, allowing a weaker model to solve a substantial fraction of benchmarks, while the LLM-alone baseline—even with the stronger model—fails on the harder protocols that require structured, incremental invariant construction.

## 7 Discussion

### 7.1 Case Study: MongoLoglessDynamicRaft (MLDR)

MongoLoglessDynamicRaft (MLDR) is a production-grade Raft variant with dynamic reconfiguration, modeling the core consensus mechanism used in MongoDB’s replication layer. The protocol maintains five state variables per server—currentTerm, state, configVersion, configTerm,

and config—and supports four actions: leader election via quorum voting, configuration propagation, dynamic reconfiguration, and term updates. The safety property requires at most one primary per term (OnePrimaryPerTerm).

**Invariant discovery.** I3DP discovers a 15-conjunct strengthening invariant (16 clauses in total, including safety) in 4,525 seconds, making MLDR the most difficult benchmark in our evaluation. The discovered conjuncts fall into three categories:

1. **Single-server constraints** (8 clauses): Well-formedness conditions scoped to individual servers, such as  $\forall s \in \text{Server} : \text{configVersion}[s] \geq 1$  and  $\forall s \in \text{Server} : \text{state}[s] = \text{Primary} \Rightarrow \text{configTerm}[s] = \text{currentTerm}[s]$ . These are discovered quickly in early IC3 rounds.

2. **Cross-server relationships** (4 clauses): Properties linking pairs of servers, including configuration uniqueness:

$$\begin{aligned} \forall s, t \in \text{Server} : \\ & \text{configVersion}[s] = \text{configVersion}[t] \\ & \wedge \text{configTerm}[s] = \text{configTerm}[t] \\ & \Rightarrow \text{config}[s] = \text{config}[t] \end{aligned}$$

and quorum overlap:

$$\begin{aligned} \forall s, t \in \text{Server} : \\ & \text{QuorumsOverlap}(\text{config}[s], \text{config}[t]) \\ & \vee \text{ConfigDisabled}(s) \vee \text{ConfigDisabled}(t) \end{aligned}$$

3. **Nested-quantifier quorum properties** (3 clauses): The hardest clauses, requiring alternating  $\forall\exists$  quantifiers over servers and quorums with cross-variable comparisons. One such clause is:

$$\begin{aligned} \forall s, t \in \text{Server} : \\ & \forall Q \in \text{QuorumsAt}(s) : \\ & \exists n \in Q : \text{currentTerm}[n] \geq \text{currentTerm}[t] \\ & \vee \text{IsNewerConfig}(n, s) \end{aligned}$$

In plain language: for any two servers  $s$  and  $t$ , every quorum in  $s$ 's configuration has a witness node  $n$  that is either at least as current as  $t$  in term number, or running a newer configuration than  $s$ . This prevents any quorum from becoming uniformly stale with respect to any other server's term—a necessary condition for reconfiguration safety. A second clause specializes this to competing primaries: if two primaries coexist with differing terms, every quorum of the lower-term primary must contain a node whose term already exceeds that primary's, preventing the stale primary from reconstituting its original election quorum. Together with a simpler quorum-level clause for single primaries, these three clauses form the crux of the reconfiguration safety argument.

**Why is MLDR hard?** The difficulty is not in obtaining counterexamples, but in eliminating residual bad states after the structural clauses have been added. The IC3 loop quickly removes most early-round counterexamples with local constraints, then faces a small residual set that can only be closed by a strong global property rather than a local state fact. The nested-quantifier clauses require the LLM to infer cross-variable relationships (e.g.,  $\text{currentTerm}[n] \geq \text{currentTerm}[t] \vee \text{IsNewerConfig}(n, s)$ ) from individual counterexample states that typically expose only a few concrete node configurations. The quorum-level existential quantification is especially challenging because the blocking clause needs a witness *set* with a specific structural property, not merely a simple universal constraint. Much of the runtime (4,525 seconds) is spent on the verification backend, where a single inductiveness query can exceed 5 minutes.

**Comparison with baselines.** Table 1 shows that Endive, IC3PO, SWISS, DuoAI, and P-FOL-IC3 all fail on MLDR. Our local Endive run does not solve this protocol, even with a manually engineered predicate pool of 38 predicates covering configuration terms, versions, quorum overlap, and the protocol's key operators. Reproducing the archived Endive result appears to require an expert-designed quantifier and predicate pool that already encodes deep protocol knowledge, precisely the kind of manual effort that I3DP eliminates.

**TLAPS proof.** For formal verification beyond the checked finite instance, we prove the discovered invariant inductive for unbounded protocol instances using TLAPS. We first simplify the 15-conjunct output by iterating from older clauses to newer ones: at each step, we remove one clause and use Apalache to re-check inductiveness on the same finite instance. Note that it is fully automatic once the inductive invariant is available. This eliminates 8 redundant clauses, yielding a minimal 7-conjunct strengthening. Such redundancy is expected: during the IC3 loop, clauses are introduced to strengthen intermediate frames and block specific bad states, but once stronger clauses are admitted, earlier weaker ones can be removed in a final simplification pass.

Recent work on agent-based system verification has found that automatically discovering inductive invariants remains beyond the reach of current LLMs [34]. However, once I3DP provides the discovered invariants, the subsequent TLAPS proofs become tractable for LLM agents. In our evaluation, a proof of over 3,000 lines (MLDR) was completed by LLM agents with minimal human intervention.

Concretely, an LLM-based agent first constructs a proof scaffold according to a set of structural rules, such as dividing proofs into per-clause and per-action obligations. A dedicated proof-advancement skill was then deployed on top of Codex's automation layer, which continuously attempts to discharge remaining proof obligations until completion. Integrating this agent-driven TLAPS workflow into a fully automated end-to-end pipeline is ongoing work (cf. § 9).

## 7.2 Soundness and Completeness

On a checked finite instance, I3DP inherits the soundness and completeness properties of the IC3 framework. For soundness, every LLM-generated clause is first screened by TLC against concrete reachable states, so no clause falsified by a reachable state can enter the IC3 frames. Moreover, the soundness guarantee of the IC3 algorithm itself ensures that no admitted clause can exclude a reachable state. For completeness, a finite instance contains only finitely many bad states; I3DP continually accumulates them and, when no further LLM-guided clause can be found, falls back to negating and combining the collected bad states. Since the complement of the bad-state set forms an inductive invariant in the finite state space, I3DP will eventually derive a correct answer.

### 7.3 Limitations

**Finite-instance validation.** The I3DP loop validates inductiveness of invariants only on finite instances. Both TLC and Apalache reason about bounded state spaces, so the IC3 fixpoint on the checked instance does not automatically generalize to arbitrary parameter configurations.

In the current work, we close this gap using TLAPS: for all 29 solved benchmarks, TLAPS proves that the discovered invariants are inductive for unbounded protocol instances, thereby establishing safety.

**Benchmark coverage.** The current 29-protocol suite covers classic distributed protocol models but may not represent the full diversity of real-world systems. Protocols with substantially different structures (such as more complex message-passing patterns, larger parameter domains, or deeply nested invariant clauses) may challenge I3DP in ways not captured by the current suite.

**Benchmark leakage.** The 29 benchmark protocols are publicly available and may have been included in the LLM’s training corpus, creating a risk of memorization. However, the invariant discovered by I3DP for MLDR is a 7-clause version, whereas the invariant available on GitHub contains 5 clauses; neither implies the other, and the key four-quantifier clauses differ substantially, suggesting that the LLM synthesizes invariants from IC3-gathered information rather than recalling pre-existing solutions. This partially alleviates the concern.

## 8 Related Work

**Automated invariant inference.** Ivy [25] provides an interactive environment for safety verification, and Feldman et al. [8] develop user-guided inference based on phase structures. Several tools target Ivy and related first-order languages to automatically infer inductive invariants for distributed protocols. I4 [21], DistAI [36], DuoAI [35], and IC3PO [9] build directly on Ivy. I4 infers invariants by model-checking finite protocol instances and generalizing the results to unbounded settings. DistAI takes a data-driven approach, enumerating candidate invariants from simulation traces and monotonically weakening them via SMT-based validation into inductive invariants; DuoAI extends it with support for existential quantification. IC3PO extends IC3 with symmetry-aware quantified reasoning, exploiting indistinguishable domain sorts to lift clauses learned from finite instances to quantified invariants. SWISS [11] performs counterexample-guided exhaustive search over quantified invariant candidates, exploiting symmetries and parallelism to reduce the search space. P-FOL-IC3 [16] extends IC3 to infer invariants with quantifier alternations on mypyvy specifications, using a breadth-first search strategy to tame the search space explosion. These tools operate in restricted first-order settings with decidable verification backends, limiting their applicability to expressive languages like TLA+.

Kondo [38] and Basilisk [39] target Dafny-based verification: Kondo introduces an invariant taxonomy separating Regular Invariants from Protocol Invariants, and Basilisk automates inductive invariant generation using provenance invariants. They extract several kinds of properties to formulate an inductive invariant, but still need a human to prove that the invariant implies the safety property and to supply hints when the invariant is too weak.

Endive [28] operates directly on TLA+ specifications using a flat counterexample-to-induction (CTI) elimination loop, with predicates generated by grammar-based enumeration from user-provided seed predicates and quantifier templates. It demonstrates that CTI-guided invariant inference is practical for TLA+ protocols without restricting the model to decidable logic fragments. Schultz et al. [27] later propose inductive proof decomposition, an interactive methodology where a human verifier incrementally builds an inductive proof graph guided by localized induction counterexamples and variable slicing. I3DP shares Endive’s emphasis on CTI-guided strengthening, but replaces flat CTI elimination with explicit IC3 frame management and grammar-based predicate generation with LLM-generated blocking clauses.

IC3 [1, 2] and earlier interpolation-based model checking [22] have been influential in hardware verification. Extensions to theory reasoning and infinite-state systems [6, 13] have broadened IC3’s applicability, while first-order quantified separators [15] address quantified invariant inference in first-order settings. Our work adopts IC3’s core control structure directly: frontier counterexamples to induction are traced backward through predecessor states in shallower frames, learned clauses are admitted up to justified frame levels, push attempts move clauses forward, and safety is proved when adjacent frames coincide. The main difference from SAT-based PDR lies in clause discovery: I3DP uses bad-state witnesses produced by Apalache together with LLM-generated protocol-level blocking clauses instead of SAT-based inductive generalization from Boolean state cubes.

Compared with EPR-based, interactive, and Dafny-based approaches, our method targets the full expressiveness of TLA+ while fitting directly into an existing TLA+/TLAPS workflow, avoiding translation or rewriting into another verification language.

**TLA+ tools and verified systems.** TLC [37] is the standard explicit-state model checker for TLA+, Apalache [17] provides symbolic model checking using SMT solvers, and TLAPS [5] supports hierarchical proof development and proof checking—together forming a mature ecosystem with significant industrial adoption [23]. Discovering inductive invariants, however, remains the primary bottleneck in the TLA+ verification workflow. Our tool attempts to complement this ecosystem by automating that step.

**LLMs for formal verification.** Recent work has explored using LLMs for formal verification tasks. For instance, for

loop invariant generation, Kamath et al. [14] prompt LLMs to generate loop invariants and then identify valid candidates using Frama-C. LaM4Inv [32] combines LLMs with bounded model checking in a query-filter-reassemble loop, and Chakraborty et al. [4] propose contrastive ranking to prioritize correct invariants among LLM-generated candidates. Clause2Inv [3] proposes a generate-combine-check framework for loop invariant synthesis. Quokka [29] adopts a simpler evaluation-oriented design that directly validates whether each LLM-generated invariant helps prove the target assertion. Pirzada et al. [26] use LLMs to generate invariants to facilitate bounded model checking.

For broader program verification, Lemur [33] integrates LLMs into automated verification; Wen et al. [30] combine LLMs with static analysis for specification synthesis.

Relatively few LLM-based verification approaches operate directly on TLA+ specifications. (A notable exception is the recent work by Zhou and Tripakis [40], which explores LLM-guided proof automation for TLA+.) One reason is that TLA+ distributed protocol models require quantified invariants over unbounded numbers of processes and complex protocol-level reasoning. I3DP targets that setting. It performs inductive invariant synthesis with an explicit IC3 controller and LLM-generated blocking clauses over TLA+ states. This enables end-to-end invariant synthesis without domain-specific predicate configuration.

## 9 Conclusion

We presented I3DP, an IC3-based TLA+ tool that combines an explicit IC3 controller with LLM-generated blocking clauses. Across 29 distributed protocols including industrial-scale benchmarks such as MLDR and one complex Paxos variant that none of the five compared tools could solve, I3DP discovers candidate invariants for all 29 protocols, without manual template design. TLAPS further confirms that the discovered invariants are inductive for unbounded protocol instances.

The key insight behind these results is that LLMs need not produce complete invariants. Rather, they are most effective as *hypothesis generators* within a structured IC3 architecture. The IC3 controller decomposes the global synthesis problem into focused per-state blocking tasks, validates every proposed clause through frame admission and forward propagation, and provides a completeness backstop when the LLM falls short. This division of labor, i.e., LLM for protocol-level conjecture and IC3 for proof discipline, underpins the robustness of our approach: even weaker (or differently trained) models can solve a substantial fraction of benchmarks because the surrounding verification structure compensates for imprecise clause proposals.

More broadly, I3DP suggests that formal verification is not necessarily an exclusive domain of verification experts. By embedding LLM reasoning inside a principled proof architecture, I3DP makes inductive-invariant discovery accessible to

protocol designers who can write a TLA+ specification but lack the expertise to craft strengthening invariants manually. We view this as a step toward making formal methods a routine of distributed systems development.

**Future work.** We plan to explore the following directions.

*Benchmark expansion.* We plan to apply I3DP to larger, more complex protocols from real engineering settings with richer state spaces, and to expand the benchmark suite with private hand-written protocols to further validate generalization beyond publicly available specifications.

*Approach strengthening.* We plan to strengthen I3DP with clause sources beyond LLMs. For example, the property-digging approach of Basilisk [39] is a promising direction.

*Automating TLAPS discharge.* Once I3DP discovers an inductive invariant, the remaining step is to prove it inductive for unbounded instances in TLAPS. We are exploring automated approaches to this proof-generation task, with the goal of closing the entire verification pipeline from invariant discovery to unbounded proof. Integrating such a proof-generation capability into I3DP would deliver a fully end-to-end verification workflow for TLA+ specifications.

## Acknowledgments

We thank our shepherd, Jay Lorch, for his insightful comments and constructive suggestions. We are grateful to the anonymous reviewers for their valuable feedback. We also thank Jiacheng Zhao for his contributions to the early stages of this project. This work is supported by the Fundamental and Interdisciplinary Disciplines Breakthrough Plan of the Ministry of Education of China (No. JYB2025XDXM118), the Frontier Technologies R&D Program of Jiangsu (BF2024059), the National Natural Science Foundation of China (No. 92582201, 62572229), and the “111 Center” (No. B26023). Taolue Chen is partially supported by overseas grants from the State Key Lab of Novel Software Technology (Nos. #KFKT2025A05 and #KFKT2026A04). Yuan Yao and Xiaoxing Ma are the corresponding authors.

## Data Availability

To ensure transparency and facilitate reproduction, we make the complete I3DP implementation, benchmark specifications, experimental results, and TLAPS proofs available at <https://github.com/SoftWiser-group/I3DP>.

## References

- [1] Aaron R. Bradley. 2011. SAT-based model checking without unrolling. In *International Conference on Verification, Model Checking, and Abstract Interpretation*. Springer, Berlin, Heidelberg, 70–87. doi:10.1007/978-3-642-18275-4\_7
- [2] Aaron R. Bradley. 2012. Understanding IC3. In *International Conference on Theory and Applications of Satisfiability Testing (SAT)*. Springer, Berlin, Heidelberg, 1–14. doi:10.1007/978-3-642-31612-8\_1
- [3] Weining Cao, Guangyuan Wu, Tangzhi Xu, Yuan Yao, Hengfeng Wei, Taolue Chen, and Xiaoxing Ma. 2025. Clause2Inv: A Generate-Combine-Check Framework for Loop Invariant Inference. *Proceedings of the ACM on Software Engineering* 2, ISSTA (2025), 1009–1030. doi:10.1145/3728920
- [4] Saikat Chakraborty, Shuvendu Lahiri, Sarah Fakhoury, Akash Lal, Madanlal Musuvathi, Aseem Rastogi, Aditya Senthilnathan, Rahul Sharma, and Nikhil Swamy. 2023. Ranking LLM-Generated Loop Invariants for Program Verification. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, Houda Bouamor, Juan Pino, and Kalika Bali (Eds.). Association for Computational Linguistics, Singapore, 9164–9175. doi:10.18653/v1/2023.findings-emnlp.614
- [5] Kaustuv Chaudhuri, Damien Doligez, Leslie Lamport, and Stephan Merz. 2008. A TLA+ proof system. In *Proceedings of the LPAR Workshop on Knowledge Exchange: Automated Provers and Proof Assistants (KEAPPA) (CEUR Workshop Proceedings, Vol. 418)*. CEUR-WS.org, Aachen, Germany, 17–37. <https://ceur-ws.org/Vol-418/paper2.pdf>
- [6] Alessandro Cimatti, Alberto Griggio, Sergio Mover, and Stefano Tonetta. 2016. Infinite-state invariant checking with IC3 and predicate abstraction. *Formal Methods in System Design* 49, 3 (2016), 190–218. doi:10.1007/s10703-016-0257-4
- [7] Niklas Eén, Alan Mishchenko, and Robert Brayton. 2011. Efficient implementation of property directed reachability. In *2011 Formal Methods in Computer-Aided Design (FMCAD)*. FMCAD Inc., Austin, TX, USA, 125–134. <https://ieeexplore.ieee.org/document/6148886/>
- [8] Yotam M. Y. Feldman, James R. Wilcox, Sharon Shoham, and Mooly Sagiv. 2019. Inferring Inductive Invariants from Phase Structures. In *Computer Aided Verification (CAV)*. Springer, Cham, 405–425. doi:10.1007/978-3-030-25543-5\_23
- [9] Aman Goel and Karem A. Sakallah. 2021. On symmetry and quantification: A new approach to verify distributed protocols. In *NASA Formal Methods: 13th International Symposium, NFM 2021, Virtual Event, May 24–28, 2021, Proceedings*. Springer, Cham, Switzerland, 131–150. doi:10.1007/978-3-030-76384-8\_9
- [10] Jim Gray and Leslie Lamport. 2006. Consensus on Transaction Commit. *ACM Transactions on Database Systems* 31, 1 (2006), 133–160. doi:10.1145/1132863.1132867
- [11] Travis Hance, Marijn Heule, Ruben Martins, and Bryan Parno. 2021. Finding Invariants of Distributed Systems: It’s a Small (Enough) World After All. In *18th USENIX Symposium on Networked Systems Design and Implementation (NSDI 21)*. USENIX Association, Virtual Event, 115–131. <https://www.usenix.org/conference/nsdi21/presentation/hance>
- [12] Chris Hawblitzel, Jon Howell, Manos Kapritsos, Jacob R. Lorch, Bryan Parno, Michael L. Roberts, Srinath Setty, and Brian Zill. 2015. IronFleet: Proving practical distributed systems correct. In *Proceedings of the 25th Symposium on Operating Systems Principles (SOSP)*. Association for Computing Machinery, New York, NY, USA, 1–17. doi:10.1145/2815400.2815428
- [13] Kryštof Hoder and Nikolaj Bjørner. 2012. Generalized property directed reachability. In *International Conference on Theory and Applications of Satisfiability Testing*. Springer, Berlin, Heidelberg, 157–171. doi:10.1007/978-3-642-31612-8\_13
- [14] Adharsh Kamath, Nausheen Mohammed, Aditya Senthilnathan, Saikat Chakraborty, Pantazis Deligiannis, Shuvendu K Lahiri, Akash Lal, Aseem Rastogi, Subhajit Roy, and Rahul Sharma. 2024. Leveraging LLMs for program verification. In *Formal Methods in Computer-Aided Design (FMCAD)* (Prague, Czech Republic). TU Wien Academic Press, Wien, 107–118. doi:10.34727/2024/isbn.978-3-85448-065-5\_16
- [15] Jason R. Koenig, Oded Padon, Neil Immerman, and Alex Aiken. 2020. First-order quantified separators. In *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation*. Association for Computing Machinery, New York, NY, USA, 703–717. doi:10.1145/3385412.3386018
- [16] Jason R. Koenig, Oded Padon, Sharon Shoham, and Alex Aiken. 2022. Inferring Invariants with Quantifier Alternations: Taming the Search Space Explosion. In *International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*. Springer, Cham, 338–356. doi:10.1007/978-3-030-99524-9\_18
- [17] Igor Konnov, Jure Kukovec, and Thanh-Hai Tran. 2019. TLA+ model checking made symbolic. *Proceedings of the ACM on Programming Languages* 3, OOPSLA, Article 123 (2019), 30 pages. doi:10.1145/3360549
- [18] Leslie Lamport. 1998. The Part-Time Parliament. *ACM Transactions on Computer Systems* 16, 2 (1998), 133–169. doi:10.1145/279227.279229
- [19] Leslie Lamport. 2001. Paxos Made Simple. *ACM SIGACT News* 32, 4 (2001), 51–58.
- [20] Leslie Lamport. 2002. *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley, Boston, MA. <https://lamport.org/tla/book.html>
- [21] Haojun Ma, Aman Goel, Jean-Baptiste Jeannin, Manos Kapritsos, Baris Kasikci, and Karem A. Sakallah. 2019. I4: Incremental Inference of Inductive Invariants for Verification of Distributed Protocols. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles*. Association for Computing Machinery, New York, NY, USA, 370–384. doi:10.1145/3341301.3359651
- [22] Kenneth L. McMillan. 2003. Interpolation and SAT-Based Model Checking. In *Computer Aided Verification (CAV)*. Springer, Berlin, Heidelberg, 1–13. doi:10.1007/978-3-540-45069-6\_1
- [23] Chris Newcombe, Tim Rath, Fan Zhang, Bogdan Munteanu, Marc Brooker, and Michael Deardouff. 2015. How Amazon Web Services Uses Formal Methods. *Commun. ACM* 58, 4 (2015), 66–73. doi:10.1145/2699417
- [24] Oded Padon, Giuliano Losa, Mooly Sagiv, and Sharon Shoham. 2017. Paxos Made EPR: Decidable Reasoning about Distributed Protocols. *Proceedings of the ACM on Programming Languages* 1, OOPSLA, Article 108 (2017), 31 pages. doi:10.1145/3140568
- [25] Oded Padon, Kenneth L. McMillan, Aurojit Panda, Mooly Sagiv, and Sharon Shoham. 2016. Ivy: Safety verification by interactive generalization. In *Proceedings of the 37th ACM SIGPLAN Conference on Programming Language Design and Implementation*. Association for Computing Machinery, New York, NY, USA, 614–630. doi:10.1145/2908080.2908118
- [26] Muhammad A. A. Pirzada, Giles Reger, Ahmed Bhayat, and Lucas C. Cordeiro. 2024. LLM-Generated Invariants for Bounded Model Checking Without Loop Unrolling. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. Association for Computing Machinery, New York, NY, USA, 1395–1407. doi:10.1145/3691620.3695512
- [27] William Schultz, Edward Ashton, Heidi Howard, and Stavros Tripakis. 2026. Interactive Safety Verification of Distributed Protocols by Inductive Proof Decomposition. In *NASA Formal Methods* (Los Angeles, CA, USA). Springer, Cham, 3–24. doi:10.1007/978-3-032-28079-4\_1
- [28] William Schultz, Ian Dardik, and Stavros Tripakis. 2022. Plain and simple inductive invariant inference for distributed protocols in TLA+. In *Proceedings of the 22nd Conference on Formal Methods in Computer-Aided Design (FMCAD)* (Trento, Italy). TU Wien Academic Press, Wien, 273–283. doi:10.34727/2022/isbn.978-3-85448-053-2\_34
- [29] Anjiang Wei, Tianran Sun, Tarun Suresh, Haoze Wu, Ke Wang, and Alex Aiken. 2025. Quokka: Accelerating Program Verification with LLMs via Invariant Synthesis. doi:10.48550/arXiv.2509.21629 arXiv:2509.21629 [cs.PL]

- [30] Cheng Wen, Jialun Cao, Jie Su, Zhiwu Xu, Shengchao Qin, Mengda He, Haokun Li, Shing-Chi Cheung, and Cong Tian. 2024. Enchanting Program Specification Synthesis by Large Language Models Using Static Analysis and Program Verification. In *Computer Aided Verification: 36th International Conference, CAV 2024, Montreal, QC, Canada, July 24–27, 2024, Proceedings, Part II* (Montreal, QC, Canada). Springer, Cham, 302–328. doi:10.1007/978-3-031-65630-9\_16
- [31] James R. Wilcox, Doug Woos, Pavel Panchekha, Zachary Tatlock, Xi Wang, Michael D. Ernst, and Thomas Anderson. 2015. Verdi: A framework for implementing and formally verifying distributed systems. In *Proceedings of the 36th ACM SIGPLAN Conference on Programming Language Design and Implementation*. Association for Computing Machinery, New York, NY, USA, 357–368. doi:10.1145/2737924.2737958
- [32] Guangyuan Wu, Weining Cao, Yuan Yao, Hengfeng Wei, Taolue Chen, and Xiaoxing Ma. 2024. LLM Meets Bounded Model Checking: Neuro-symbolic Loop Invariant Inference. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering* (Sacramento, CA, USA) (ASE '24). Association for Computing Machinery, New York, NY, USA, 406–417. doi:10.1145/3691620.3695014
- [33] Haoze Wu, Clark Barrett, and Nina Narodytska. 2024. Lemur: Integrating Large Language Models in Automated Program Verification. In *The Twelfth International Conference on Learning Representations*. 2968–2978. <https://openreview.net/forum?id=Q3YaCghZNt>
- [34] Chenyuan Yang, Natalie Neamtu, Chris Hawblitzel, Jacob R. Lorch, and Shan Lu. 2025. VeruSAGE: A Study of Agent-Based Verification for Rust Systems. doi:10.48550/arXiv.2512.18436 arXiv:2512.18436 [cs.OS]
- [35] Jianan Yao, Runzhou Tao, Ronghui Gu, and Jason Nieh. 2022. DuoAI: Fast, automated inference of inductive invariants for verifying distributed protocols. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. USENIX Association, Carlsbad, CA, 485–501. <https://www.usenix.org/conference/osdi22/presentation/yao>
- [36] Jianan Yao, Runzhou Tao, Ronghui Gu, Jason Nieh, Suman Jana, and Gabriel Ryan. 2021. DistAI: Data-driven automated invariant learning for distributed protocols. In *15th USENIX Symposium on Operating Systems Design and Implementation (OSDI 21)*. USENIX Association, 405–421. <https://www.usenix.org/conference/osdi21/presentation/yao>
- [37] Yuan Yu, Panagiotis Manolios, and Leslie Lamport. 1999. Model checking TLA+ specifications. In *Correct Hardware Design and Verification Methods: 10th IFIP WG 10.5 Advanced Research Working Conference, CHARME'99*. Springer, Berlin, Heidelberg, 54–66. doi:10.1007/3-540-48153-2\_6
- [38] Tony Nuda Zhang, Travis Hance, Manos Kapritsos, Tej Chajed, and Bryan Parno. 2024. Inductive Invariants That Spark Joy: Using Invariant Taxonomies to Streamline Distributed Protocol Proofs. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. USENIX Association, Santa Clara, CA, 837–853. <https://www.usenix.org/conference/osdi24/presentation/zhang-nuda>
- [39] Tony Nuda Zhang, Keshav Singh, Tej Chajed, Manos Kapritsos, and Bryan Parno. 2025. Basilisk: Using Provenance Invariants to Automate Proofs of Undecidable Protocols. In *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*. USENIX Association, Boston, MA, 1–17. <https://www.usenix.org/conference/osdi25/presentation/zhang-tony> Best Paper Award.
- [40] Yuhao Zhou and Stavros Tripakis. 2026. Towards Language Model Guided TLA+ Proof Automation. In *Formal Methods* (Tokyo, Japan) (Lecture Notes in Computer Science, Vol. 16556). Springer, Cham, 619–640. doi:10.1007/978-3-032-26204-2\_32